RELAZIONE ESERCITAZIONE 2 – ALESSIO ATTILIA

Come input avevo un dataset multi classe e l'obiettivo è stato quello di confrontare due strategie di approccio di apprendimento delle reti neurali.

- Decomposizione One vs All
- Approccio diretto multi classe

SCELTE SUL CODICE

1. Csv.reader

Prima di scrivere il codice ho analizzato i file che avrebbero costituito il dataset e ho notato che non contenevano intestazioni. Se avessi utilizzato librerie come *pandas* con il comando **pd.read_csv()** senza parametri aggiuntivi, la prima riga sarebbe stata interpretata come intestazione, causando una lettura errata e la perdita della prima istanza di dati. Per evitare questo problema, ho implementato una logica che conta il numero di colonne nella prima riga e genera nomi fittizi per gli attributi quando non sono presenti intestazioni.

```
primo_giro = True
# se la riga è vuota saltala
for row in reader:
    if not row: continue

    # Se è la prima riga e non ho ancora i nomi degli attributi
    if primo_giro and not nomi_attributi:
        # conto le colonne -1
        num_colonne = len(row) - 1  # Escludo la classe
        # creo nomi
        nomi_attributi = [f"att_{i}" for i in range(num_colonne)]
        primo_giro = False
```

2. Generazione formato arff

I file di input sono stati convertiti in formato ARFF (*Attribute-Relation File Format*). Nelle righe di codice seguenti viene definito il nome del dataset tramite la direttiva @RELATION, che si comporta da identificatore e appare come prima linea nel file ARFF:

```
f.write(f"@RELATION {nome_relazione} \n\n")
```

La definizione della variabile target è un dettaglio cruciale:

```
f.write(f"@ATTRIBUTE {att} REAL\n")
```

Le parentesi graffe indicano che la classe è **nominale** e per il nostro esercizio binaria ({positive,negative}). Questa specifica è fondamentale per evitare che i parser interpretino le etichette come stringhe generiche o numeri interi, il che porterebbe a trattare il problema come regressione anziché classificazione. La sezione @DATA conclude la struttura, contenendo i vettori delle feature e la classe: `f.write(f"@DATA\n")`

3. One vs All

La realizzazione di questo punto nel codice è stata realizzata tramite una trasformazione dinamica delle etichette. Se una classe target viene mappata come positive, le altre come negative. Ho dovuto inserire uno shuffle per rimescolare i dati poiché i dati originali erano ordinati per classe. Ipotetici scenari:

- **No shuffling**: la rete potrebbe vedere prima solo "positiva", poi solo "negativa" e di conseguenza imparare dei pattern che portano all'errore
- **Shuffling:** la rete vede un mix di classi ad ogni passo

```
random.shuffle(dataset_temporaneo)
dataset_binari_memoria[target_class] = dataset_temporaneo
```

4. Pipeline e pre-processing

L'architettura del classificatore necessità la standardizzazione dei dati tramite StandardScaler. Come noto dalla teoria delle Reti Neurali, l'algoritmo di retro propagazione è estremamente sensibile alla scala delle features in input: attributi con ordini di grandezza differenti possono deformare la superficie di errore, rallentando drasticamente la discesa del gradiente. L'incapsulamento dello scaler e del classificatore in una Pipeline (dove l'output del primo processo diventa input del secondo) è stato un passaggio fondamentale. Se avessi calcolato media e varianza sull'intero dataset *prima* della cross-validation, avrei commesso un grave errore. In quello scenario le informazioni statistiche del set di test avrebbero macchiato il training set, portando il modello a ottenere prestazioni fortuitamente alte in fase di validazione, ma scadenti su nuovi dati reali.

```
mlp_pipeline = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(20,), max_iter=800, random_state=SEED)
)
```

5. Architettura della rete

Prima di progettare la rete neurale, ho analizzato i file per comprendere la quantità e la natura dei dati disponibili. In base a questo ho definito la topologia della rete seguendo un criterio di equilibrio tra complessità e capacità di generalizzazione. Ho scelto **20 neuroni** nel livello nascosto, un numero modesto che cerca di evitare fenomeni di overfitting mantenendo la rete efficiente. Durante i test ho sperimentato un incremento del numero di neuroni, ma l'aumento della complessità ha portato a un peggioramento delle prestazioni.

6. Robustezza della validazione: Stratified K-Fold

Ho scelto la strategia di validazione Stratified K-Fold Cross-Validation (k=5) rispetto ad un semplice Train/Test per alcune motivazioni. Nella strategia One vs All il dataset è sbilanciato (argomento trattato nel 3 punto). Se io avessi fatto un campionamento casuale semplice avrei potuto ottenere dei fold di test privi di esempi positivi. Necessitando di estrema precisione, la stratificazione impone dei vincoli statistici sulle

proporzioni delle classi presenti nel dataset originale. Questo approccio evita distorsioni dovute a squilibri e garantisce un "modesto" calcolo della metrica Recall.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
```

7. Cross-Val Predict vs Score

I dettagli fanno la differenza e me ne servivano maggiori soprattutto sugli errori. Sapere la media delle accuratezze dei fold senza dire dove il modello sbaglia non mi era utile e non mi dava l'idea di ciò che stato trattando. È stato il motivo principale dopo alcuni tentativi per preferire **cross_val_predict** alla **cross_val_score**. Dai dati, riportati nelle pagine finali, è emerso che gli errori erano principalmente nelle classi 2 e 3. Avendo più metriche a disposizione ho potuto maturare anche un'analisi più dettagliata.

8. Valutazioni finali

L'elevata accuratezza complessiva conferma la coerenza delle scelte implementate, in particolare l'uso di **StandardScaler** e della validazione incrociata con **StratifiedKFold**. Senza normalizzazione, la rete avrebbe potuto incontrare difficoltà di convergenza, soprattutto con una topologia relativamente semplice (20 neuroni nello strato nascosto). L'approccio **multi-classe**, in linea teorica, è preferibile e i risultati ottenuti lo dimostrano parzialmente anche in questo esercizio. Dal punto di vista probabilistico, un modello multi-classe impone che la somma delle probabilità sia pari a 1: se la rete aumenta la probabilità della classe 3, deve necessariamente ridurre quella delle altre classi. Questo vincolo riduce ambiguità e migliora la coerenza delle decisioni.

Nell'approccio **One-vs-All**, ogni classificatore binario opera in modo indipendente. In presenza di un punto ambiguo vicino al confine tra classe 2 e classe 3, la rete dedicata alla classe 2 potrebbe assegnare una probabilità di 0.6 e quella per la classe 3 una probabilità di 0.7, generando un conflitto e potenziali falsi positivi. Questo evidenzia come la gestione congiunta delle probabilità nel modello multi-classe sia più robusta rispetto alla somma di decisioni binarie indipendenti.

La classe 1 risulta linearmente separabile con estrema precisione mentre le altre due possiamo dire che non hanno confini ben distinti anche se i dati ottenuti sono comunque molto alti in accuratezza. La rete multi classe apprende direttamente le features discriminanti che distinguono la classe 2 dalla 3, ma la differenza è molto irrisoria. Un dataset bilanciato sicuramente è stato d'aiuto per ottenere questi risultati.

Modello	Accuracy
Binario – Classe 1 vs Tutte	1.00
Binario – Classe 2 vs Tutte	0.95
Binario – Classe 3 vs Tutte	0.96
Multi-classe	0.97

Di seguito ci sono le tabelle per tutti gli scenari con la leggenda per le intestazioni:

- **Precision** → percentuale delle predizioni positive che sono corrette.
- **Recall** → Percentuale di veri positivi trovati rispetto a tutti i positivi reali.
- **F1-score** → media armonica tra precision e recall, misura l'equilibrio tra errori.
- **Support** → numero reale di istanze appartenenti a una classe nel test.
- **Accuracy** → percentuale totale di predizioni corrette su tutte le istanze.
- **Macro avg** → media delle metriche tra le classi, tutte pesano uguale.
- **Weighted avg** → media ponderata secondo il numero di istanze per classe (support).

MODELLO BINARIO 1

Classe	Precision	Recall	F1-score	Support
negative	1.00	1.00	1.00	100
positive	1.00	1.00	1.00	50
Accuracy	**1.00**			**150**
Macro avg	1.00	1.00	1.00	150
Weighted avg	1.00	1.00	1.00	150

MODELLO BINARIO 2

Classe	Precision	Recall	F1-score	Support
negative	0.97	0.96	0.96	100
positive	0.92	0.94	0.93	50
Accuracy	**0.95**			**150**
Macro avg	0.95	0.95	0.95	150
Weighted avg	0.95	0.95	0.95	150

MODELLO BINARIO 3

Classe	Precision	Recall	F1-score	Support
negative	0.98	0.96	0.97	100
positive	0.92	0.96	0.94	50
Accuracy	**0.96**			**150**
Macro avg	0.95	0.96	0.96	150
Weighted avg	0.96	0.96	0.96	150

MODELLO MULTICLASSE

Classe	Precision	Recall	F1-score	Support
1	1.00	1.00	1.00	50
2	0.98	0.94	0.96	50
3	0.94	0.98	0.96	50
Accuracy	**0.97**			**150**
Macro avg	0.97	0.97	0.97	150

Classe	Precision	Recall	F1-score	Support
Weighted avg	0.97	0.97	0.97	150